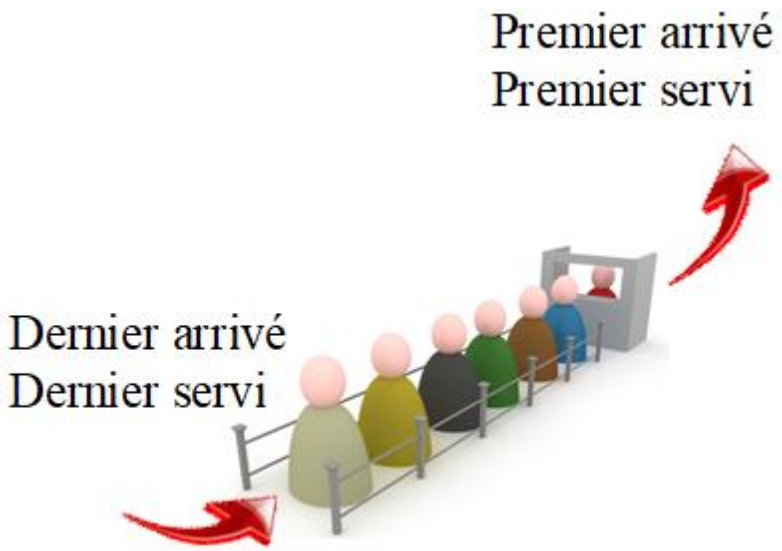
Structures de données linéaires. Les files. Correction.

En informatique, une file (queue en anglais) est une structure de données basée sur le principe « Premier entré, premier sorti », en anglais FIFO (First In, First Out), ce qui veut dire que les premiers éléments ajoutés à la file seront les premiers à être récupérés.
Le fonctionnement ressemble à une file d'attente : les premières personnes à arriver sont les premières personnes à sortir de la file.



Voici quelques exemples d'usage courant d'une file :
• En général, on utilise des files pour mémoriser temporairement des transactions qui doivent attendre pour être traitées.
• Les serveurs d'impression, qui doivent traiter les requêtes dans l'ordre dans lequel elles arrivent, et les insèrent dans une file d'attente (ou une queue).
• Certains moteurs multitâches, dans un système d'exploitation, qui doivent accorder du temps-machine à chaque tâche, sans en privilégier aucune.

Pour implémenter une structure de file, on a besoin d'un nombre réduit d'opérations de bases :
• « enfiler » : ajoute un élément dans la file. Terme anglais correspondant : « enqueue ».
• « défiler » : renvoie le prochain élément de la file, et le retire de la file. Terme anglais correspondant : « dequeue ».
• « vide » : renvoie vrai si la file est vide, faux sinon.
• « remplissage » : renvoie le nombre d'éléments dans la file.

La structure de file est un concept abstrait. Comment la réaliser dans la pratique ?
Voici plusieurs implémentations possibles. L'idée principale étant que les fonctions de bases pourront être utilisées indépendamment de l'implémentation choisie.

I) <u>IMPLEMENTATION - METHODE 1</u> :

Nous utiliserons une simple liste pour représenter la pile.
Là encore nous utiliserons append(x) et pop(0) pour réaliser les méthodes "enfiler" et "défiler".
Voici les fonctions de base :

```
def file():
    #retourne une file vide
    return []

def vide(f):
    ''' renvoie True si la file est vide
```

```
    False sinon'''
    return f==[]

def enfiler(f,x):
    #ajoute x à la file f''
    return f.append(x)

def defiler(f):
    #enlève et renvoie le premier élément de la file
    assert not vide(f), "file vide"
    return f.pop(0)
```

À faire 1 :
Tester les instructions suivantes :

```
f=file()
for i in range(5):
    enfiler(f,2*i)
print(f)
a=defiler(f)
print(a)
print(f)
print(vide(f))
```

Correction :

```
f=file()
```

Création d'une file vide.

```
for i in range(5):
    enfiler(f,2*i)
```

Remplissage de la file par les nombres pairs de 0 à 8.

```
print(f)
```

Affichage de la file : [0, 2, 4, 6, 8]

```
a=defiler(f)
```

Suppression du premier élément de la file, et renvoi de celui-ci : 0

```
print(a)
```

Affichage de l'élément supprimé : 0.

```
print(f)
```

Affichage de la file modifiée : [2, 4, 6, 8].

```
print(vide(f))
```

Teste si la file est vide : retour : False.

Exercice 1 :
Réaliser les fonctions taille(f) et sommet(f) qui retournent respectivement la taille de la file et le sommet de la file (le premier à sortir) sans le supprimer.

```
def taille(f):
    pass

def sommet(f):
    pass
```

Correction :

```
def taille(f):
```

```
    return len(f)

def sommet(f):
    return f[0]
```

II) IMPLÉMENTATION - MÉTHODE 2 :

Nous allons créer une classe File pour implémenter cette structure.

À faire 2 :
En vous inspirant de ce que l'on a vu pour la classe Pile(), réaliser cette implémentation

```
class File:
    ''' classe File
    création d'une instance File avec une liste
    '''
    def __init__(self):
        "Initialisation d'une file vide"
        ...........

    def vide(self):
        "teste si la file est vide"
         ...........

    def defiler(self):
        "défile"
        ..........................

    def enfiler(self,x):
        "enfile"
        ..................
```

Correction :

```
class File:
    ''' classe File
    création d'une instance File avec une liste
    '''
    def __init__(self):
        "Initialisation d'une file vide"
        self.L = []

    def vide(self):
        "teste si la file est vide"
        return self.L == []

    def defiler(self):
        "défile"
        assert not self.vide(), "File vide"
        return self.L.pop(0)

    def enfiler(self,x):
        "enfile"
        self.L.append(x)
```

Exercice 2 :
Écrire les instructions permettant de :
• créer une file,
• la remplir avec les entiers 0,2,4,6,8,
• la faire afficher,
• "défiler" la file en faisant afficher l'élément récupéré.

Correction :

```
f=File()
for i in range(5):
    f.enfiler(2*i)
print(f.L)
a=f.defiler()
print(a)
```

Réaliser les méthodes taille(self) et sommet(self) qui retourne respectivement la taille de la file et le sommet de la file(le premier à sortir..) sans le supprimer.

```
def taille(self):
    pass

def sommet(self):
    pass
```

Correction :

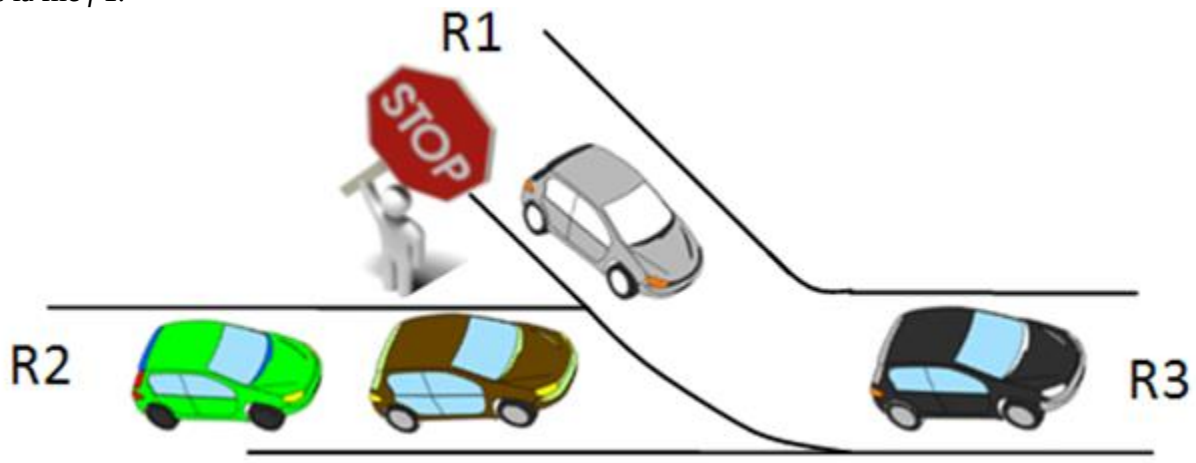
```
Dans la classe File :
def taille(self):
        return len(self.L)

    def sommet(self):
        return self.L[0]

# en dehors de la classe :
print(f.taille())
print(f.sommet())
```

III) <u>EXEMPLE - CROISEMENT ROUTIER</u> :

Pour simuler un croisement routier, à sens unique, on utilise 3 files *f* 1, *f* 2 et *f* 3 représentant respectivement les voitures arrivant sur des routes R1 et R2, et les voitures partant sur la route R3.
La route R2 a un STOP. Les voitures de la file *f* 2 ne peuvent avancer que s'il n'y a aucune voiture sur la route R1, donc dans la file *f* 1.



On souhaite écrire un algorithme qui simule le départ des voitures sur la route R3, modélisée par la file *f* 3.
_ Dans la file *f* 1 on représentera la présence d'une voiture par le nombre 1 et l'absence de voiture par 0.
_ Dans la file *f* 2 on représentera la présence d'une voiture par le nombre 2 et l'absence de voiture par 0.
_ On n'utilisera que les méthodes enfiler, defiler, sommet et vide.
_ On testera l'algorithme sur *f* 1 : tête <–[0, 1, 1, 0, 1]<– queue.
_ On testera l'algorithme sur *f* 2 : tête <–[0, 2, 2, 2, 0, 2, 0]<– queue.
_ Le résultat attendu : *f* 3 : tête <–[0, 1, 1, 2, 1, 2, 2, 0, 2, 0]<– queue.

<u>Question 1</u> :

Que doit faire l'algorithme si les deux sommets des files sont à 0 ?
Il doit donner 0.

Question 2 :
Que doit faire l'algorithme si le sommet de *f* 1 est à 1 et celui de *f* 2 à 2 ?
Il doit donner 1.

Question 3 :
Que doit faire l'algorithme si le sommet de *f* 1 est à 1 et celui de *f* 2 à 0 ?
Il doit donner 1.

Question 4 :
Que doit faire l'algorithme si le sommet de *f* 1 est à 0 et celui de *f* 2 à 2 ?
Il doit donner 2.

Question 5 :
Que doit faire l'algorithme si l'une des deux files est vide ?
Il doit donner ce qui reste de l'autre file.

Question 6 :
Compléter l'algorithme qui modélise ce carrefour. On utilisera une fonction croisement(f1,f2) qui prend en paramètres deux files *f* 1 et *f* 2 et qui retourne une file *f* 3 contenant la file *f* 3 des voitures sur la route R3.
On utilise trois tableaux f1, f2 et f3 qui seront utilisées comme des files.

```
def Croisement_routier(f1,f2) :
   n1, n2 = longueur de f1, longueur de f2
   i1, i2 = 1, 1

   tant que f1 et f2 ne sont pas vides :
      si le sommet de f1 = 1
         defiler f1 et enfiler f3 avec cet élément
         augmenter i1 de 1
         si le sommet de f2 = 0
            defiler f2
            ............

      sinon
         si le sommet de f2 = 2
            defiler f2 et enfiler f3 avec cet élément
            augmenter i2 de 1
            ...............
            augmenter i1 de 1
         sinon
            defiler f2 et enfiler f3 avec cet élément
            augmenter i2 de 1
            defiler f1
            ...............

   # on complète f3 avec le reste de la pile qui n'est pas vide
   tant que i1 <= n1
      defiler f1 et enfiler f3 avec cet élément
      augmenter i1 de 1

   tant que i2 <= n2
      ....................
      ....................

   retourner f3
```

Correction :

```
def Croisement_routier(f1,f2) :
    n1, n2 = longueur de f1, longueur de f2
    i1, i2 = 1, 1

    tant que f1 et f2 ne sont pas vides :
        si le sommet de f1 = 1
            defiler f1 et enfiler f3 avec cet élément
            augmenter i1 de 1
            si le sommet de f2 = 0
                defiler f2
                augmenter i2 de 1

        sinon
            si le sommet de f2 = 2
                defiler f2 et enfiler f3 avec cet élément
                augmenter i2 de 1
                defiler f1
                augmenter i1 de 1
            sinon
                defiler f2 et enfiler f3 avec cet élément
                augmenter i2 de 1
                defiler f1
                augmenter i1 de 1

    # on complète f3 avec le reste de la pile qui n'est pas vide
    tant que i1 <= n1
        defiler f1 et enfiler f3 avec cet élément
        augmenter i1 de 1

    tant que i2 <= n2
        defiler f2 et enfiler f3 avec cet élément
        augmenter i2 de 1

    retourner f3
```

Exercice 3 :
Réaliser le programme et tester le dans différents scénarios.

```
def file():
    #retourne une file vide
    return []

def vide(f):
    ''' renvoie True si la file est vide
    False sinon'''
    return f==[]

def enfiler(f,x):
    #ajoute x à la file f''
    return f.append(x)

def defiler(f):
    #enlève et renvoie le premier élément de la file
    assert not vide(f), "file vide"
    return f.pop(0)

def Croisement_routier(f1,f2) :
    n1, n2 = len(f1), len(f2)
    i1, i2 = 1, 1
```

```
    while (not vide(f1)) and (not(vide(f2))):
        if f1[0] == 1:
            enfiler(f3, defiler(f1))
            i1 += 1
            if f2[0] == 0:
                defiler(f2)
                i2 += 1
        else:
            if f2[0] == 2:
                enfiler(f3, defiler(f2))
                i2 += 1
                defiler(f1)
                i1 += 1

            else:
                enfiler(f3, defiler(f2))
                i2 += 1
                defiler(f1)
                i1 += 1

    while i1 <= n1:
        enfiler(f3, defiler(f1))
        i1 += 1

    while i2 <= n2:
        enfiler(f3, defiler(f2))
        i2 += 1

    return f3

f1 = [0,1,1,0,1]
f2 = [0,2,2,2,0,2,0]
f3 = []

print(Croisement_routier(f1,f2))
```

Retourne : [0, 1, 1, 2, 1, 2, 2, 0, 2, 0]

Prolongement possible
Complexifier la situation en imaginant des croisements successifs de trois ou quatre routes avec des STOP et des priorités à droite...

IV) EXERCICES :

Pour les exercices qui suivent, on utilise les fonctions définies pour les piles et les files. On peut aussi travailler avec les méthodes correspondantes.

```
def empiler(p,x):
    "Ajoute l'élément x à la pile p"
    p.append(x)

def depiler(p):
    "dépile et renvoie l'élément au sommet de la pile p"
    return p.pop()

def enfiler(f,x):
    #ajoute x à la file f"
    return f.append(x)

def defiler(f):
```

```
    #enlève et renvoie le premier élément de la file
    return f.pop(0)
```

Exercice 4 :
On dispose d'une file, écrire une fonction qui renvoie la file inversée (l'élément de la tête sera situé à la queue et ainsi de suite). On utilisera seulement les méthodes associées aux piles et aux files.
Correction :

```
def file_inversion(f):
    n = len(f)
    file_inversee = []
    if n != 0:
        for i in range(n):
            x = depiler(f)
            enfiler(file_inversee,x)
    return file_inversee
```

Exercice 5 :
On dispose d'une file contenant des entiers, écrire une fonction qui renvoie une file où on aura séparé les nombres pairs des impairs.
Correction :

```
def file_pairs_impairs(f):
    n = len(f)
    file_reponse = []
    if n != 0:
        for i in f:
            if i % 2 == 0:
                enfiler(file_reponse,i)
        for i in f:
            if i % 2 != 0:
                enfiler(file_reponse,i)
    return file_reponse
```

Exercice 6 :
On dispose d'une pile contenant des entiers, écrire une fonction ou une méthode qui renvoie une pile où l'on aura supprimé le premier élément entré.
Correction :

```
def pile_suppr_premier(f):
    pile_reponse = f
    defiler(file_reponse)
    return pile_reponse
```

Exercice 7 :
On dispose d'une file contenant des entiers, écrire une fonction ou une méthode qui renvoie une file où l'on aura supprimé le dernier élément entré.
Correction :

```
def file_suppr_dernier(f):
    file_reponse = f
    depiler(file_reponse)
    return file_reponse
```